

Kubernetes

VS Code extensions

- <https://marketplace.visualstudio.com/items?itemName=redhat.vscode-yaml>
- <https://marketplace.visualstudio.com/items?itemName=ipedrazas.kubernetes-snippets>

minikube

```
# linux
> curl -LO
https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-
amd64
> sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-
amd64

# macOS - x86
> curl -LO
https://github.com/kubernetes/minikube/releases/latest/download/minikube-darwin-
amd64
> sudo install minikube-darwin-amd64 /usr/local/bin/minikube

# macOS - arm
> curl -LO
https://github.com/kubernetes/minikube/releases/latest/download/minikube-darwin-
arm64
> sudo install minikube-darwin-arm64 /usr/local/bin/minikube

# check the status of cluster
> minikube status

# start minikube cluster
> minikube start

# stop the cluster
# - it will not remove the cluster / virtual machine
> minikube stop

# delete the cluster
> minikube delete

# get the minikube dashboard (control plane)
> minikube dashboard

# ssh to minikube virtual machine
> minikube ssh

# add minikube kubectl in bashrc
> vim ~/.bashrc
```

```
# add the following line
> alias kubectl="minikube kubectl --"

# expose a service outside the minikube cluster
# > minikube service <service name>
```

cluster

```
# get the cluster information
> kubectl cluster-info

# get detailed information about the cluster
> kubectl cluster-info dump

# get all api versions supported
> kubectl api-versions

# get all resource mappings
> kubectl api-resources
```

nodes

```
# get the list of nodes
> kubectl get nodes
```

namespace

```
# get the list of namespaces
> kubectl get namespaces
> kubectl get ns

# get all the resources created from a ns
# > kubectl get all -n <ns name>
> kubectl get all -n kube-system

# create a namespace
> kubectl create ns myns

# delete a selected namespace
# note: everything inside ns will also get deleted
```

```
# > kubectl delete ns <ns name>
> kubectl delete ns myns
```

pod

```
# get the list of pods
> kubectl get pods

# get the list of pods with more details
> kubectl get pods -o wide

# run application inside a pod (imperative command)
# > kubectl run <pod name> --image <image>
> kubectl run httpd --image httpd

# create a new pod inside myns namespace
# > kubectl run <pod name> --image <image>
> kubectl run httpd --image httpd -n myns

# create a pod using yaml file
# > kubectl create -f <file name>

# get details of a selected pod
# > kubectl describe pod <pod name>
> kubectl describe pod httpd

# execute a command inside a pod
# > kubectl exec -it <pod name> -- <command>
> kubectl exec -it mysql -- date

# get logs generated by a pod
> kubectl logs <pod name>
```

replica set

- used to create multiple replicas of an application

```
# get the list of replica sets
> kubectl get replicaset
> kubectl get rs

# create a replica set
# > kubectl create -f <replica set yaml file>
```

deployment

- uses replica set internally to create multiple replicas
- can update the versions easily in the future

```
# get the list of deployments
> kubectl get deployments
> kubectl get deploy

# create a deployment
# > kubectl create -f <deployment yaml file>

# delete a deployment
# > kubectl delete deploy <deployment name>

# update a deployment
# > kubectl rollout deploy <deploy name>
```

service

- acts as a load balancer
- types
 - ClusterIP
 - used to balance the load internal to the cluster
 - does not expose the application outside the cluster
 - NodePort:
 - used to expose the application outside the cluster
 - LoadBalancer:
 - used to create a load balancer on cloud

```
# get the list of services
> kubectl get services
> kubectl get svc

# create a service using yaml definition
# > kubectl create -f <yaml file>

# get the details of selected service
# > kubectl describe service <service name>

# get the Endpoints for the service
# > kubectl describe service <service name> | grep Endpoints

# delete a service
```

```
# > kubectl delete service <service name>
```

persistent volume

persistent volume claim

config map

- collection of key-value pairs
- used to create environment variables inside the pod
- used to keep non-sensitive config values like urls, ports etc.

```
# get the list of config maps
> kubectl get configmap
> kubectl get cm

# get details of selected config map
> kubectl describe cm <cm name>
```

secret

- collection of key-value pairs
- used to create environment variables inside the pod
- used to keep sensitive config params like API Key, password, tokens etc.

```
# get the list of secrets
> kubectl get secrets
> kubectl get secret

# get details of selected secret
> kubectl describe secret <secret name>
```

prometheus

```
# add the prometheus helm repo
> helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

# update the prometheus repo
> helm repo update

# install prometheus from the helm repo
> helm install prometheus prometheus-community/prometheus

# update the prometheus service's type to NodePort
> kubectl edit service/prometheus-server
```

grafana

```
# install the helm repo grafana
> helm repo add grafana https://grafana.github.io/helm-charts

# update the repo
> helm repo update

# install grafana
> helm install grafana grafana/grafana

# update the grafana service's type to NodePort
> kubectl edit service/grafana

# find the grafana admin password
# copy admin password from the file
> kubectl edit secret/grafana

# decode the password to string using base64
# > echo <password> | base64 --decode

# dashboard: 18283, 1860
```

helm

- package manager for kubernetes
- helm chart: package which contains the application component definitions to be deployed in k8s cluster

installation

```
# macOS
> brew install helm

# ubuntu
> sudo apt-get install curl gpg apt-transport-https --yes
> curl -fsSL https://packages.buildkite.com/helm-linux/helm-debian/gpgkey | gpg --
dearmor | sudo tee /usr/share/keyrings/helm.gpg > /dev/null
> echo "deb [signed-by=/usr/share/keyrings/helm.gpg]
https://packages.buildkite.com/helm-linux/helm-debian/any/ any main" | > sudo tee
/etc/apt/sources.list.d/helm-stable-debian.list
> sudo apt-get update
> sudo apt-get install helm

# verify installation
> helm --version
```

using helm

```
# create a new chart
# > helm create <chart name>
> helm create myapp

# get the list of helm charts installed on your machine
> helm list

# install helm chart
# > helm install <chart name> <path>
> cd myapp
> helm install myapp .

# uninstall a helm chart
# > helm uninstall <chart name>
> helm uninstall myapp
```